

Recurrent Neural Network

Muhammad Yazid Supriadi, S.Kom, M.Kom
STT Terpadu Nurul Fikri

Tujuan Perkuliahan

- Pengenalan terkait konsep Recurrent Neural Network
- Mengenali aplikasi RNN dalam NLP
- Implementasi RNN dalam NLP



Sequential data

Vs

Non-Sequential Data

Data berurutan (Sequential Data) adalah data yang:

- Memiliki ukuran input yang bervariasi
- Urutan input penting
- Informasi sebelumnya memengaruhi informasi di masa depan
- Harus diproses langkah demi langkah secara berurutan

1. Panjang Input Variabel

- Panjang urutan dapat berubah
- Contoh: Kalimat A: “saya suka artificial intelligence”
Kalimat B: “saya sangat suka belajar artificial intelligence”
Kedua kalimat memiliki panjang yang berbeda.

2. Urutan Penting

- Mengubah urutan mengubah makna.
- Contoh: "Saya suka ayam goreng"
“ayam goreng menyukai saya”
Kata-kata sama, urutan berbeda, makna berbeda.

3. Input Sebelumnya Mempengaruhi Output di Masa Depan

- Data sebelumnya memengaruhi prediksi di masa depan
- Informasi masa lalu membantu memprediksi hasil berikutnya
- Contoh: Harga saham sebelumnya membantu memprediksi harga besok

4. Konteks / history Itu Penting

- Model membutuhkan memori dari informasi sebelumnya
- Pemahaman bergantung pada konteks historis
- Contoh: Dalam sebuah kalimat, kata-kata sebelumnya membantu menentukan makna kata berikutnya

Non Sequential Data

Data yang tidak berurutan memiliki karakteristik:

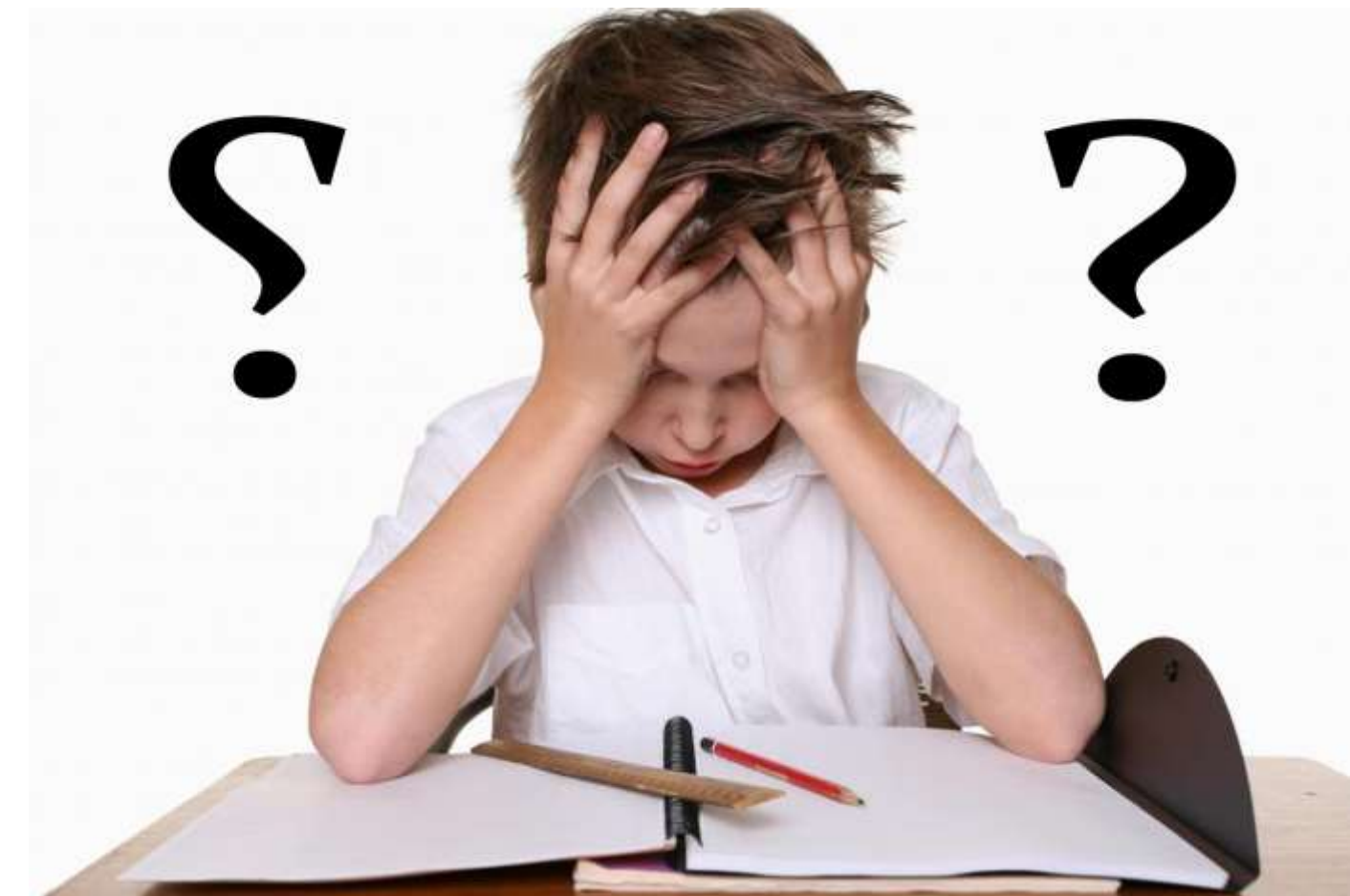
- Tidak bergantung urutan
- Tiap fitur dianggap berdiri sendiri
- Biasanya ukuran input tetap

Contoh non sequential data:

- Data tabular Excel
- Data customer
- Data gambar
- Data tinggi/berat/umur
- Dataset klasifikasi biasa

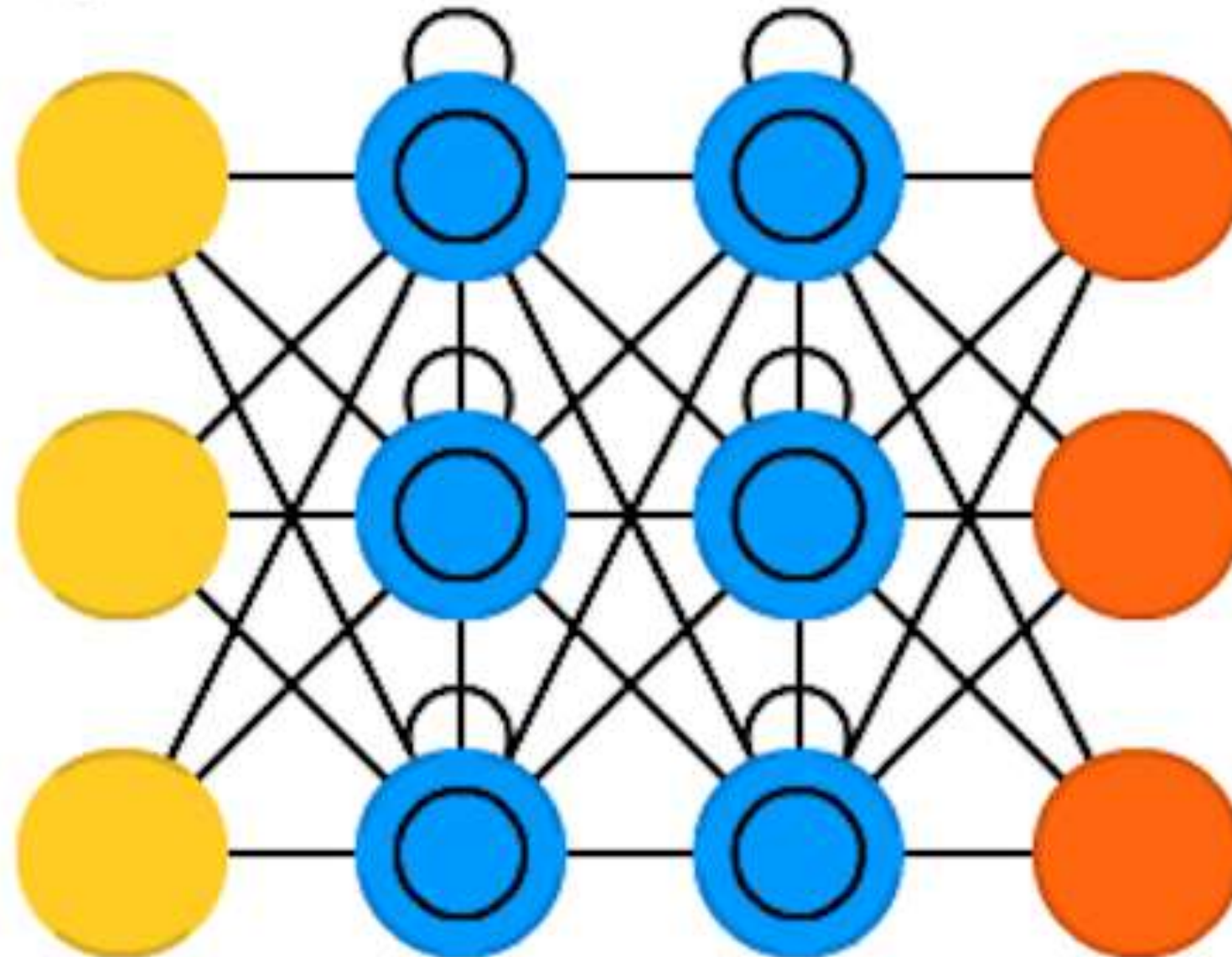
Jaringan Saraf Tradisional Kesulitan dengan Data Berurutan

- Jaringan Neural Tradisional memerlukan ukuran input yang tetap
- Setiap fitur input harus memiliki dimensi yang konsisten
- Bobot dan bias dipelajari berdasarkan struktur input yang tetap
- Input diproses secara independen tanpa mengingat input sebelumnya



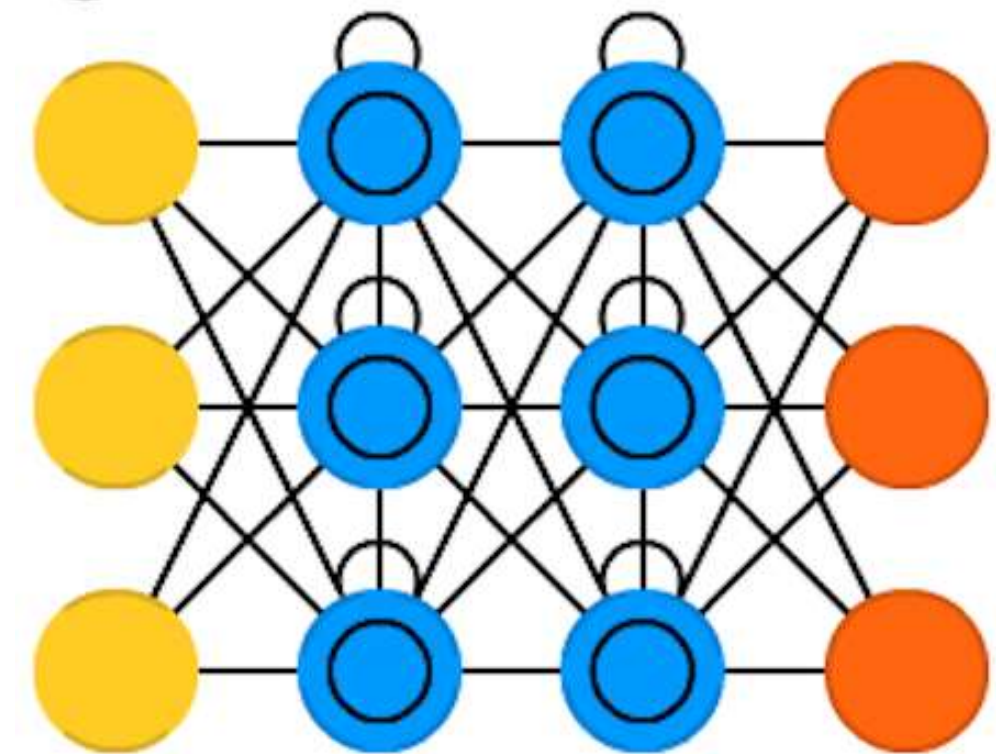
- Data berurutan berubah seiring waktu
- Panjang input bisa berbeda untuk setiap kasus
- Urutan data penting
- Informasi sebelumnya mempengaruhi prediksi di masa depan
- Contoh:
 - Prediksi harga saham
 - Peramalan cuaca
 - Analisis keuangan
 - Pemrosesan Bahasa Alami (NLP)

Recurrent Neural Network

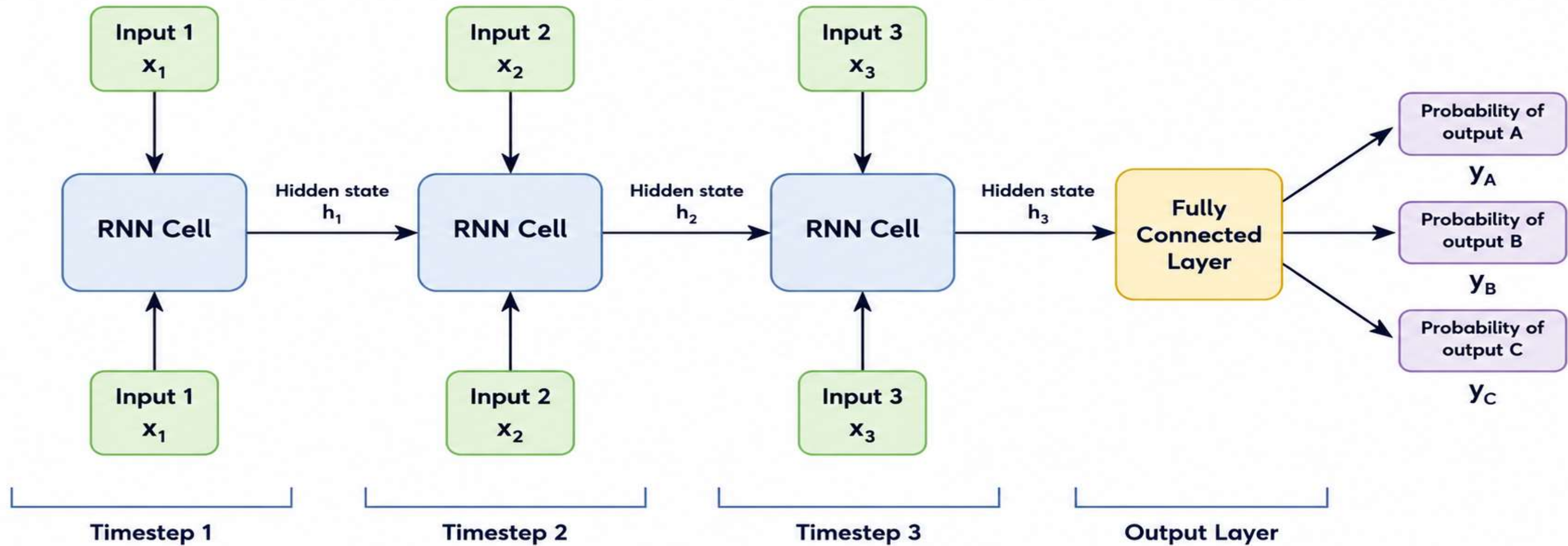


Recurrent Neural Network

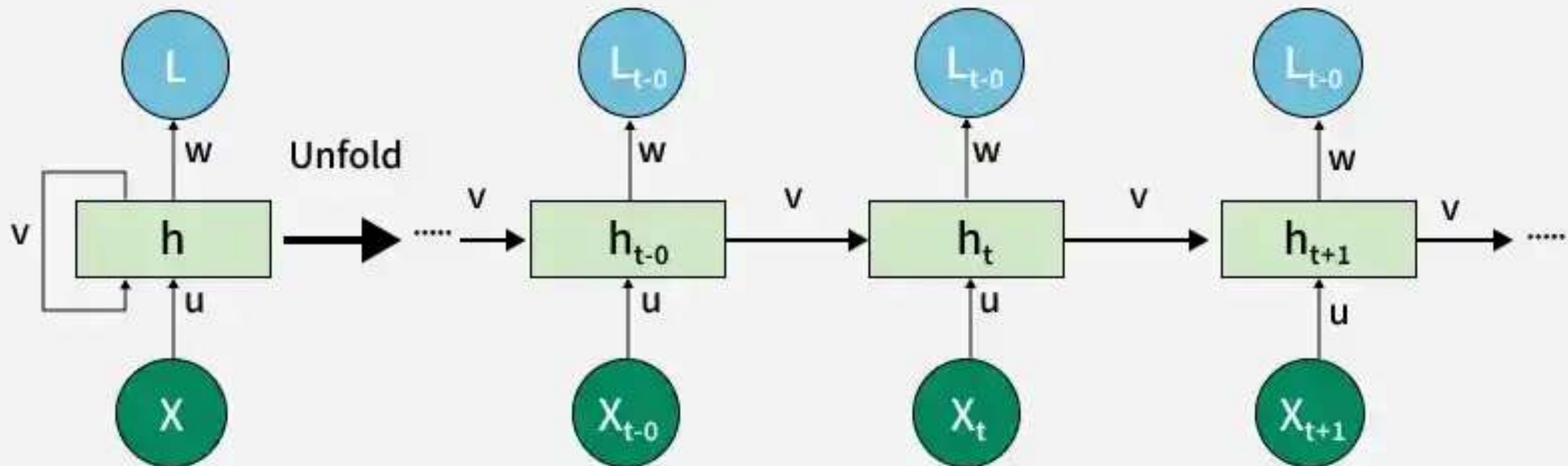
- Recurrent Neural Network (RNN) adalah jenis jaringan syaraf buatan yang dirancang untuk pemrosesan data berurutan.
- Berbeda dengan traditional feedforward neural network, RNN memiliki koneksi yang membentuk siklus terarah, yang memungkinkan mereka untuk mempertahankan keadaan tersembunyi yang menangkap informasi dari input sebelumnya.
- Hal ini membuat RNN sangat cocok untuk tugas-tugas yang melibatkan data deret waktu, natural language processing, dan pengenalan suara.



RNN Architecture (Unrolled)

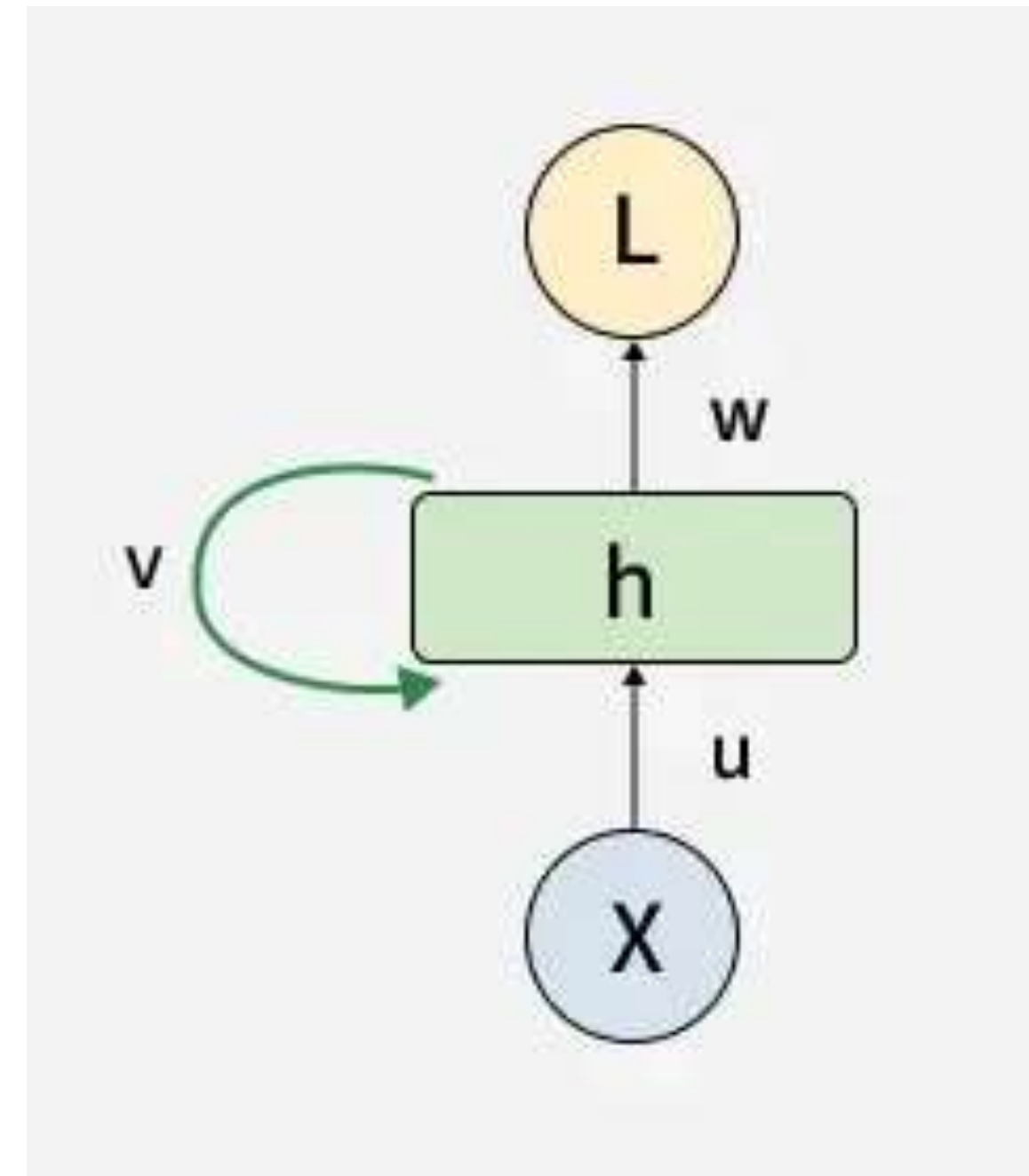


Working of Recurrent Neural Network



Recurrent Neurons

- Komponen pemrosesan fundamental dalam RNN disebut **Recurrent Unit**.
- Sebuah **Recurrent Unit** mempertahankan **Hidden state** yang menyimpan informasi dari langkah waktu sebelumnya.
- **Hidden state** ini berfungsi sebagai memori jaringan.



RNN Unrolling

Apa itu RNN Unrolling?

- RNN unrolling adalah proses memperluas struktur recurrent network ke dalam beberapa timestep.
- Setiap timestep direpresentasikan sebagai RNN cell yang terpisah.
- Walaupun terlihat terpisah, semua cell menggunakan weight dan parameter yang sama.

Cara Kerja Unrolling

Contoh sequence:

- Kata 1 $\rightarrow$ Kata 2 $\rightarrow$ Kata 3 $\rightarrow$ Kata 4

Saat proses unrolling:

- Setiap kata diproses secara berurutan
- Hidden state diteruskan ke timestep berikutnya

Flow:

- $x_1 \rightarrow h_1 \rightarrow x_2 \rightarrow h_2 \rightarrow x_3 \rightarrow h_3$

Informasi mengalir secara sequential antar timestep.

Mengapa Unrolling Penting?

Unrolling membantu kita untuk:

- Memvisualisasikan aliran informasi temporal
- Memahami propagasi hidden state
- Melatih RNN pada data sequential

Backpropagation Through Time (BPTT)

- Setelah proses unrolling:
 - Error dipropagasikan kembali melewati seluruh timestep
 - Proses ini disebut: **Backpropagation Through Time (BPTT)**
 - Tujuan:
 - Mengupdate weight dan bias
 - Mempelajari dependency dari timestep sebelumnya
 - Meningkatkan performa prediksi sequence

Mengapa Unrolling Penting?

Unrolling membantu kita untuk:

- Memvisualisasikan aliran informasi temporal
- Memahami propagasi hidden state
- Melatih RNN pada data sequential

Backpropagation Through Time (BPTT)

- Setelah proses unrolling:
 - Error dipropagasikan kembali melewati seluruh timestep
 - Proses ini disebut: **Backpropagation Through Time (BPTT)**
 - Tujuan:
 - Mengupdate weight dan bias
 - Mempelajari dependency dari timestep sebelumnya
 - Meningkatkan performa prediksi sequence

Alur Singkat Perhitungan RNN

1. Input sequence masuk

Data diproses satu per satu berdasarkan urutan timestep.

2. Hitung hidden state

Input saat ini digabung dengan hidden state sebelumnya.

3. Generate output

Hidden state digunakan untuk menghasilkan prediksi/output.

4. Teruskan hidden state

Hidden state baru dikirim ke timestep berikutnya sebagai memory.

5. Hitung loss/error

Prediksi dibandingkan dengan label sebenarnya.

6. Backpropagation Through Time (BPTT)

Error dipropagasikan mundur ke seluruh timestep.

7. Update weight dan bias

Parameter diperbarui untuk memperbaiki performa model.

8. Ulangi training

Proses diulang sampai model belajar pola sequence dengan baik.

Formula Perhitungan RNN

1. HIDDEN STATE (MEMORY)

$$h_t = f(\underbrace{W_{xh}x_t}_{\text{Input ke hidden}} + \underbrace{W_{hh}h_{t-1}}_{\text{Hidden sebelumnya ke hidden}} + \underbrace{b_h}_{\text{Bias hidden}})$$

- h_t Hidden state pada timestep t (MEMORI)
- x_t Input pada timestep t
- h_{t-1} Hidden state dari timestep sebelumnya
- W_{xh} Bobot input-to-hidden
- W_{hh} Bobot hidden-to-hidden (shared across time)
- b_h Bias pada hidden layer
- f Fungsi aktivasi (contoh: tanh)

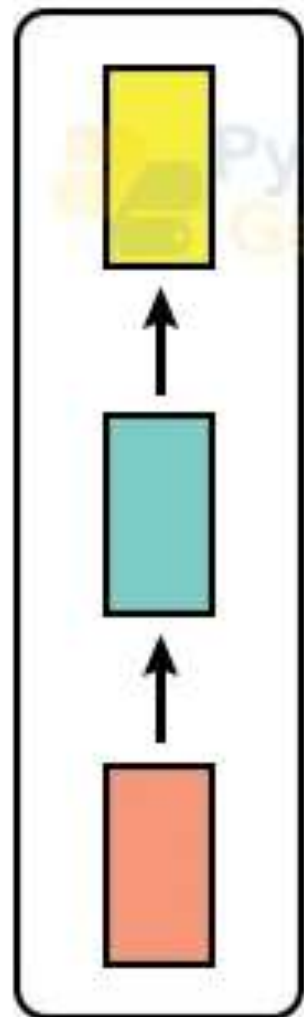
2. OUTPUT

$$y_t = g(\underbrace{W_{hy}h_t}_{\text{Hidden ke output}} + \underbrace{b_y}_{\text{Bias output}})$$

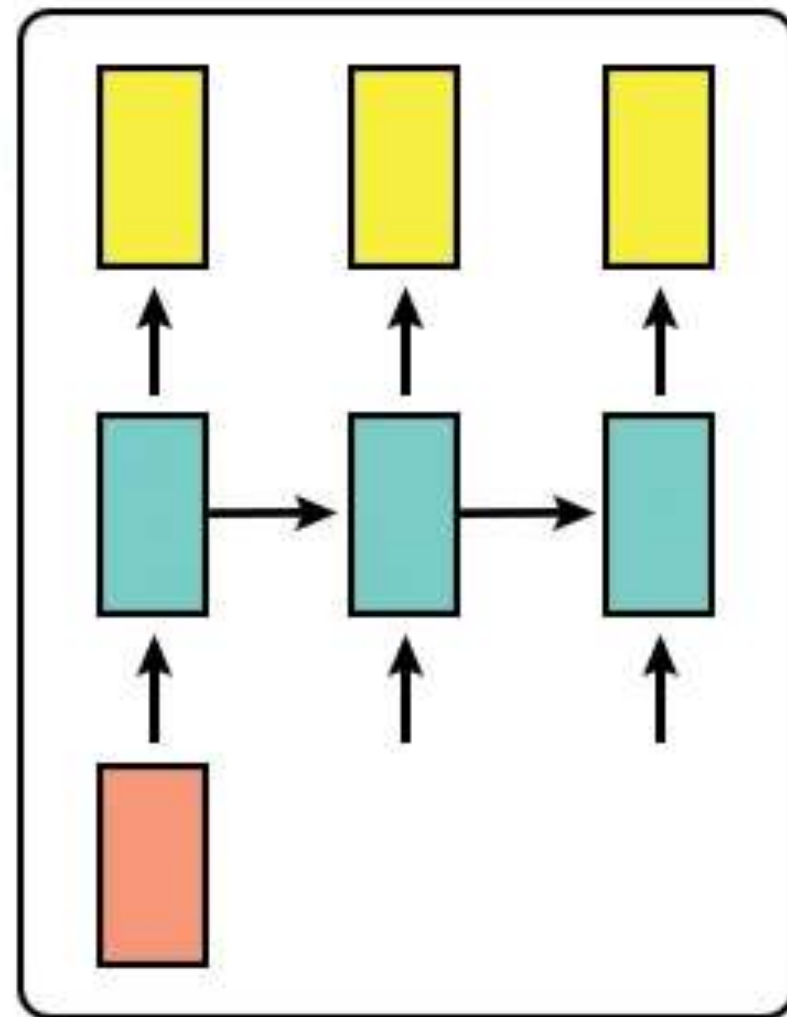
- y_t Output pada timestep t
- h_t Hidden state terbaru
- W_{hy} Bobot hidden-to-output
- b_y Bias pada output layer
- g Fungsi aktivasi (contoh: softmax untuk klasifikasi)

Types of RNN

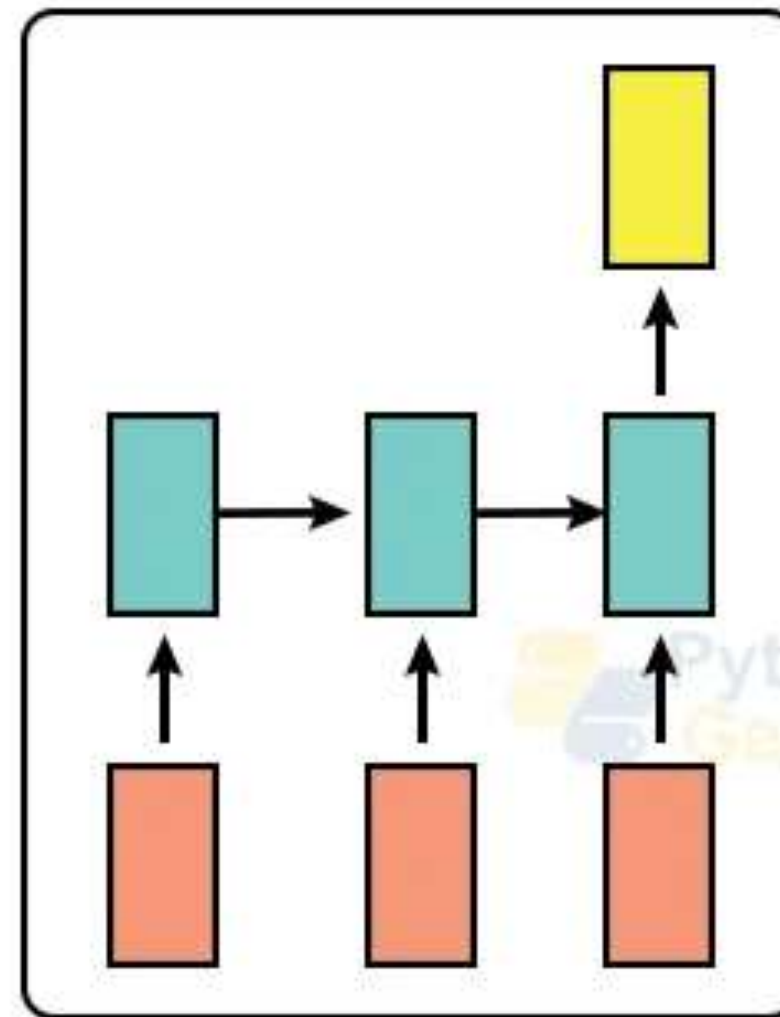
one to one



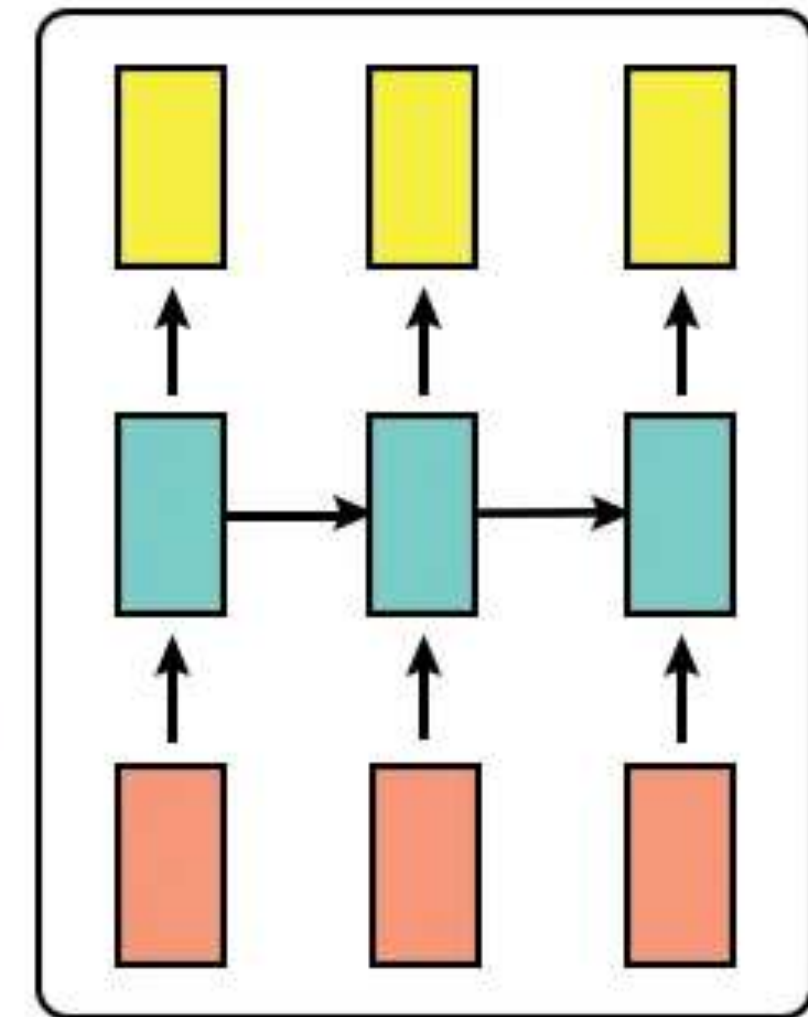
one to many



many to one



many to many



One to One

Konsep

Satu input menghasilkan satu output.

Cara Kerja

Model menerima satu teks/dokumen sebagai input, lalu menghasilkan satu prediksi atau label.

Contoh dalam NLP yaitu spam classification

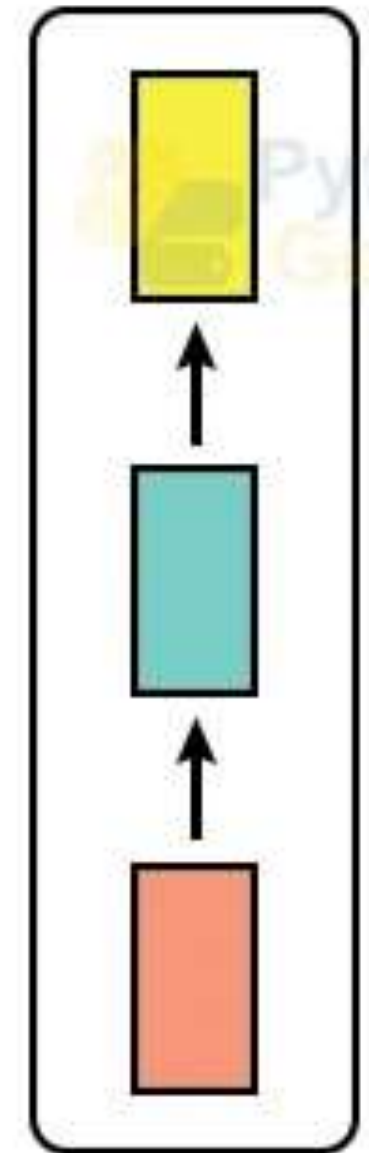
Input:

- “Congratulations! You won a free iPhone!”

Output:

- Spam

one to one



One to Many

Konsep

Satu input menghasilkan sequence output.

Cara Kerja

Model menerima satu input awal, lalu menghasilkan kata demi kata secara berurutan.

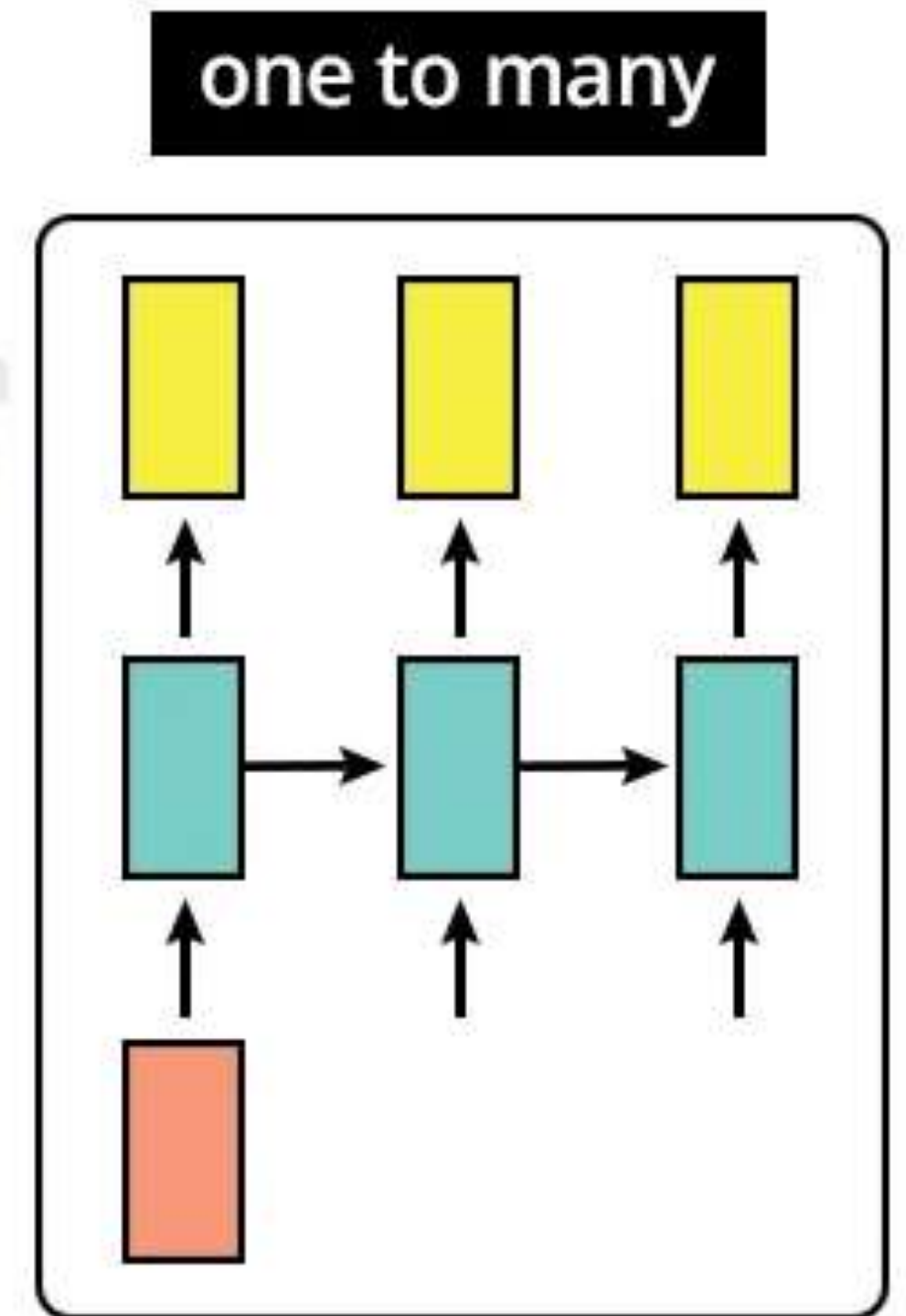
Contoh pengaplikasian dalam NLP seperti Text Generation

Input:

- “Artificial Intelligence”

Output:

- “Artificial Intelligence is transforming the future of technology.”



Many to One

Konsep

Sequence input menghasilkan satu output.

Cara Kerja

Model membaca seluruh urutan kata, memahami konteksnya, lalu menghasilkan satu prediksi akhir.

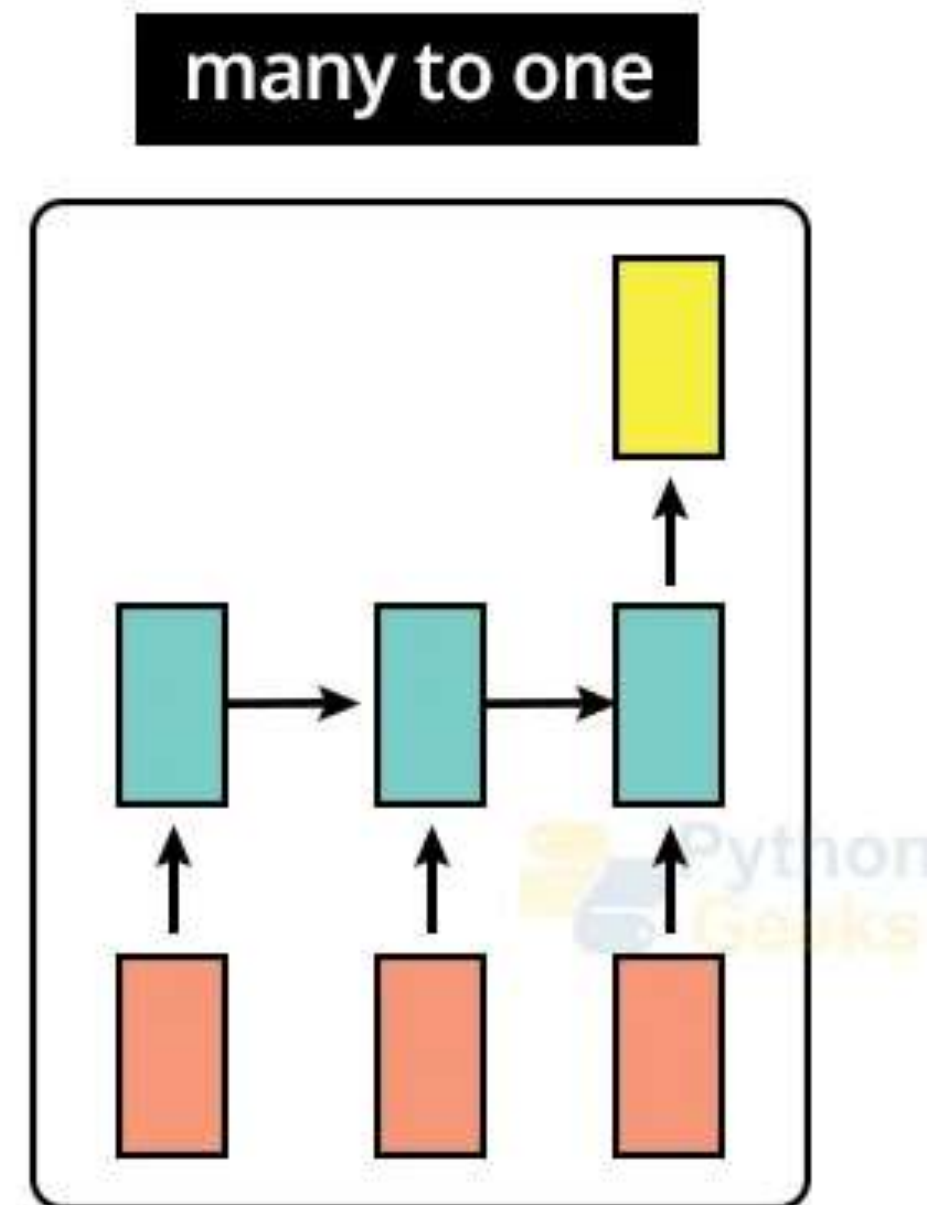
Contoh aplikasi dalam NLP adalah Sentiment Analysis

Input:

- “Film ini sangat bagus dan ceritanya menarik.”

Output:

- Positive



Many to Many

Konsep

Setiap input memiliki output pada setiap timestep.

Cara Kerja

Model memproses kata satu per satu dan memberikan label untuk setiap kata.

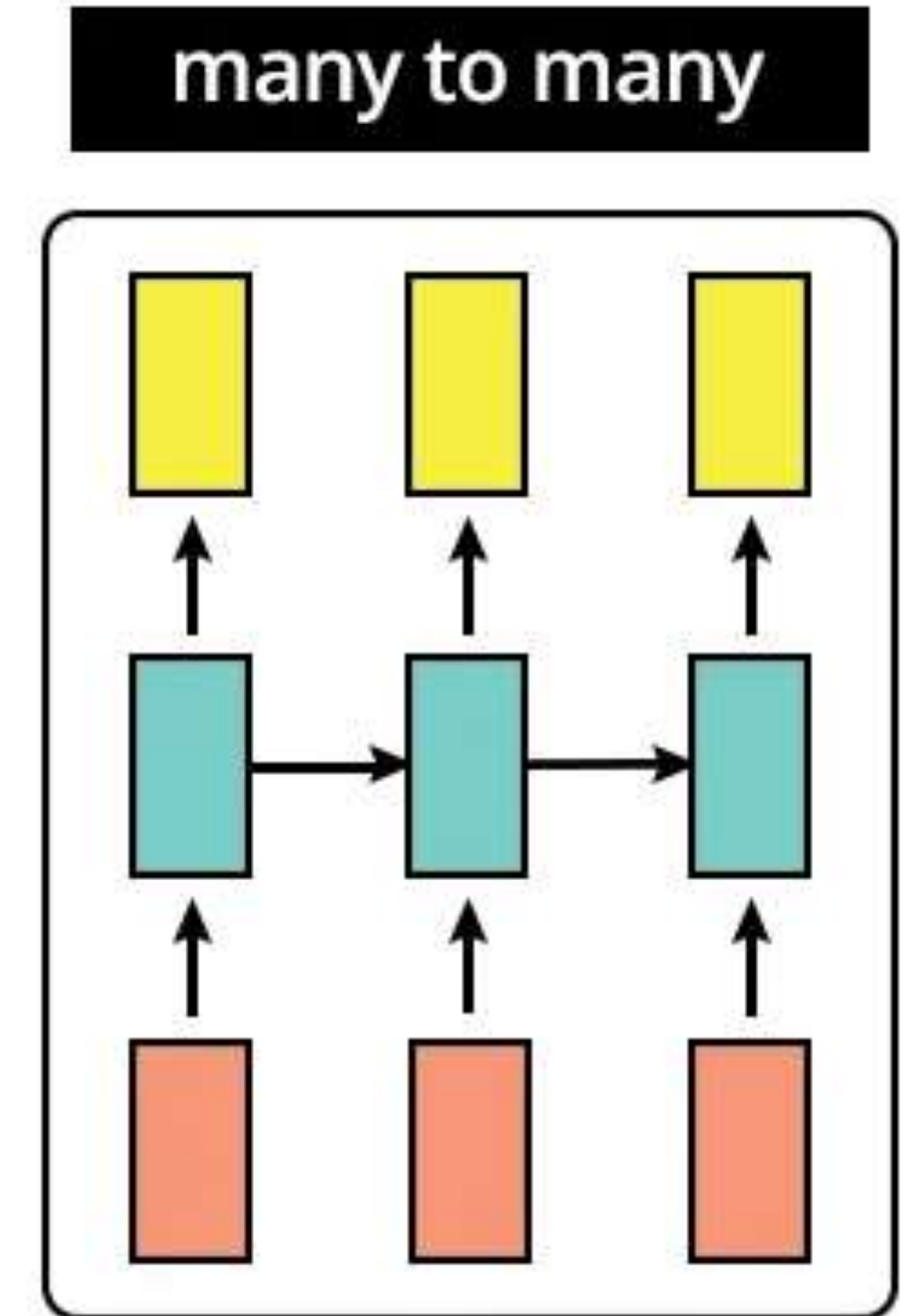
Contoh aplikasi dalam NLP adalah POS Tagging

Input:

- “Saya suka AI”

Output:

- Saya → Pronoun
- suka → Verb
- AI → Noun



Keunggulan RNN

- Mampu memproses data sequential seperti teks, audio, dan time series.
- Memiliki hidden state (memory) untuk menyimpan informasi dari timestep sebelumnya.
- Dapat memahami konteks dan hubungan antar urutan data.
- Mampu menangani input dengan panjang yang berbeda-beda.
- Dapat mempelajari dependency temporal antara data masa lalu dan data saat ini.
- Menggunakan parameter sharing sehingga lebih efisien dalam pembelajaran sequence.
- Sangat cocok digunakan pada aplikasi NLP seperti sentiment analysis, machine translation, chatbot, dan text generation.
- Menjadi dasar pengembangan model sequence modern seperti LSTM dan GRU.

Kelemahan RNN

Vanishing Gradient Problem

- Saat proses backpropagation, nilai gradient semakin mengecil pada setiap timestep.
- Akibatnya:
 - Update weight menjadi sangat kecil
 - Model sulit belajar dependency jangka panjang
 - Informasi dari timestep awal perlahan “hilang”

Dampak

- RNN sulit mengingat konteks yang jauh.
- Performa menurun pada sequence panjang.

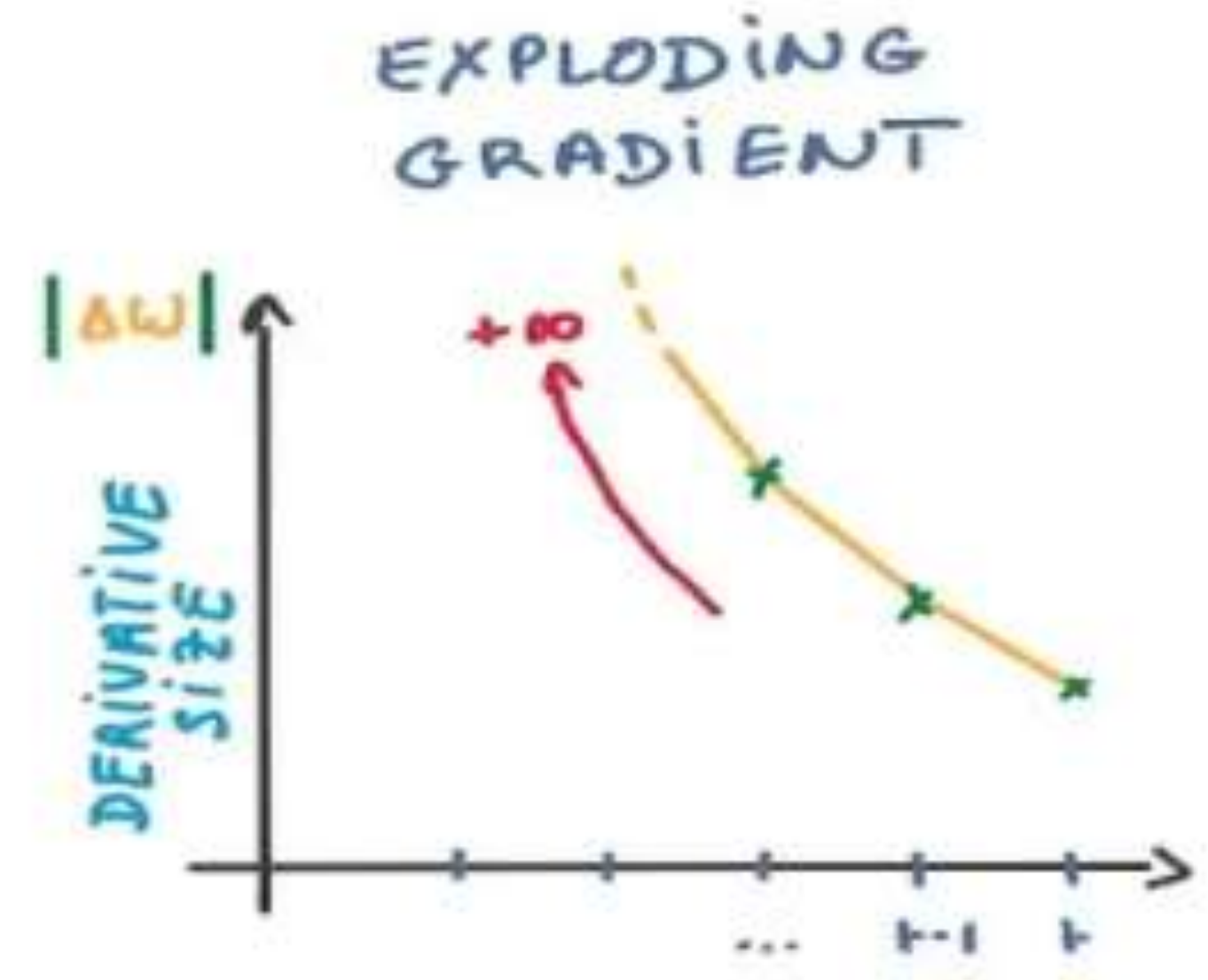


2. Exploding Gradient Problem

- Gradient dapat bertambah sangat besar secara tidak terkontrol saat backpropagation.
- Akibatnya:
 - Update weight menjadi terlalu besar
 - Training menjadi tidak stabil
 - Loss dapat berubah drastis atau menjadi NaN

Dampak

- Model sulit konvergen
- Training menjadi gagal atau tidak konsisten



3. Training Lambat

- RNN memproses data secara sequential (satu per satu timestep).
- Tidak dapat melakukan parallel processing secara optimal.

Dampak

- Training membutuhkan waktu lebih lama.
- Kurang efisien untuk dataset besar.

4. Memory Terbatas

- Hidden state memiliki kapasitas terbatas untuk menyimpan informasi.

Dampak

- Tidak semua informasi penting dapat dipertahankan.
- Model dapat lupa konteks sebelumnya.

5. Sensitif terhadap Urutan Data

- Perubahan urutan input dapat mengubah hasil secara signifikan.

Dampak

- Noise atau kesalahan urutan dapat mempengaruhi performa model.

6. Sulit Belajar Dependency Kompleks

- RNN dasar kesulitan memahami hubungan yang sangat jauh atau kompleks dalam sequence.

Contoh NLP

- Hubungan antar kata yang berjauhan dalam paragraf panjang.

Thank you guys!